Is Inflation Outpacing Wage Growth?

Photo by Tima Miroshnichenko from Pexels

Remember when milk was one dollar a bottle? Remember when it cost one dollar to watch a movie? Remember when you could buy a new car for one dollar? Ah, the good ole days. How many of us have heard friends/family/ourselves complain about how everything has grown except their salary? Is this really true, are prices really increasing more than our salaries?

Is our purchasing power really decreasing, or is it just an illusion?

https://en.wikipedia.org/wiki/Purchasing_power

This notebook will try to examine inflation vs wage growth in the U.S. by looking at this dataset: https://www.kaggle.com/bls/bls, specifically:

1. *CPI data from the cpi_u table (inflation)*
2. *Wage data from the employment_hours_earnings (CES) table (wage growth)*

This is a Google BigQuery dataset and is updated monthly.

```
import numpy as np
import pandas as pd
pd.set_option('display.max_rows', 100)
pd.set_option('display.max_colwidth', None)
```

```
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
# from plotly.offline import init_notebook_mode
# init_notebook_mode(connected=True)

import datetime
import textwrap

from google.cloud import bigquery

import warnings
warnings.filterwarnings('ignore')

import plotly.io as pio
pio.templates.default = "plotly_white"
# pio.renderers.default = 'kaggle'
# pio.renderers
```

```
# function for safe queries (based on Kaggle SQL course)
def querytodf(query):
    dry_run_config = bigquery.QueryJobConfig(dry_run=True)

    # API request - dry run query to estimate costs
    dry_run_query_job = client.query(query, job_config=dry_run_config)
    print("This query will process {} bytes.".format(dry_run_query_job.total_bytes_processed))

    # Only run the query if it's less than 1 GB
    ONE_GB = 1000*1000*1000
    safe_config = bigquery.QueryJobConfig(maximum_bytes_billed=ONE_GB)

    # Set up the query (will only run if it's less than 1 GB)
    safe_query_job = client.query(query, job_config=safe_config)

    return safe_query_job.to_dataframe()

# get 'bls' dataset
client = bigquery.Client()
dataset_ref = client.dataset("bls", project="bigquery-public-data")
dataset = client.get_dataset(dataset_ref)
```

```
Using Kaggle's public dataset BigQuery integration.
```

Getting to know CPI-U, CES

We need to explore the structure of the CPI-U and CES data in order to understand how to build our queries.

Both the CPI data and CES data each have a directory on the bls.gov website with supplementary documentation.

CPI-U: Consumer Price Index, All Urban Consumers

from https://download.bls.gov/pub/time.series/cu/cu.txt:

> The Consumer Price Index (CPI) is a statistical measure of change, over time, of the prices of goods and services in major expenditure groups--such as food, housing, apparel, transportation, and medical care--typically purchased by urban consumers. Essentially, it compares the cost of a sample "market basket" of goods and services in a specific month relative to the cost of the same "market basket" in an earlier reference period. This reference period is designated as the base period.

- https://www.bls.gov/cpi/
- https://download.bls.gov/pub/time.series/cu/

CES: Current Employment Statistics, National

from https://download.bls.gov/pub/time.series/ce/ce.txt:

> The Current Employment Statistics (CES) program provides estimates of employment,
> hours, and earnings information on a national basis and in considerable industry
> detail. The Bureau of Labor Statistics collects payroll data each month from a sample of business and government establishments in all nonfarm activities.

- https://www.bls.gov/ces/
- https://download.bls.gov/pub/time.series/ce/

For simplicity's sake, we'll use seasonally adjusted data unless otherwise mentioned.

https://en.wikipedia.org/wiki/Seasonality

Examine CPI data

Column data + first 5 rows:

```
# check column info for CPI-U table
table_ref = dataset_ref.table("cpi_u")
table = client.get_table(table_ref)
display(table.schema)

# show first 5 rows
client.list_rows(table, max_results=5).to_dataframe()
```

```
[SchemaField('series_id', 'STRING', 'REQUIRED', 'Code identifying the specific series', (), None),
 SchemaField('year', 'INTEGER', 'NULLABLE', 'Identifies year of observation', (), None),
 SchemaField('period', 'STRING', 'NULLABLE', 'Identifies period for which data is observed. M01 = January, M02 = February….M13 = Annual Average', (), None),
 SchemaField('value', 'FLOAT', 'NULLABLE', 'Price index for item', (), None),
 SchemaField('footnote_codes', 'STRING', 'NULLABLE', 'Identifies footnote for the data series', (), None),
 SchemaField('survey_abbreviation', 'STRING', 'NULLABLE', 'Code identifying the survey', (), None),
 SchemaField('seasonal_code', 'STRING', 'NULLABLE', 'Code identifying whether the data are seasonally adjusted. S = Seasonally Adjusted, U = Unadjusted', (), None),
 SchemaField('periodicity_code', 'STRING', 'NULLABLE', 'Frequency of data observation. R = Monthly, S = Semi-Annual', (), None),
 SchemaField('area_code', 'STRING', 'NULLABLE', 'Unique code used to identify a specific geographic area. Full area codes found here: http://download.bls.gov/pub/time.series/cu/cu.area', (), None),
 SchemaField('area_name', 'STRING', 'NULLABLE', 'Name of specific geographic area.', (), None),
 SchemaField('item_code', 'STRING', 'NULLABLE', 'Identifies item for which data observations pertain. Full item codes found here: http://download.bls.gov/pub/time.series/cu/cu.item', (), None),
 SchemaField('item_name', 'STRING', 'NULLABLE', 'Full names of items.', (), None),
 SchemaField('date', 'DATE', 'NULLABLE', 'Specifies period in date format. M01 -> [year]-01-01...., M12 -> [year]-12-01, M13 -> [year]-12-31, Q01 -> [year]-01-15...., Q04 -> [year]-10-15, Q05 -> [year]-12-31, S01 -> [year]-06-30, S02 -> [year]-12-30, S03 -> [year]-12-31, A01 -> [year]-12-31', (), None)]
```

	series_id	year	period	value	footnote_codes	survey_abbreviation	seasonal_code	periodicity_
0	CUSR0000SA0	1997	M01	159.4	None	CU	S	
1	CUSR0000SA0	1997	M02	159.7	None	CU	S	
2	CUSR0000SA0	1997	M03	159.8	None	CU	S	
3	CUSR0000SA0	1997	M04	159.9	None	CU	S	
4	CUSR0000SA0	1997	M05	159.9	None	CU	S	

The cu.series table gives more information about each set of time series data in the CPI table. This file is not included in the Kaggle dataset so we'll import it from the bls.gov website.

The table is quite large, so we'll filter the data:

- area_code: 0000 (average of all U.S. cities)
- seasonal: S (seasonally adjusted)
- periodicity_code: R (monthly observations)

Note: There is also data for different areas of the U.S. I may add this in future versions of this notebook, but for now we'll just look at average of all cities.

```
# 'cu.series' from https://download.bls.gov/pub/time.series/cu/cu.series
url = 'https://download.bls.gov/pub/time.series/cu/cu.series'
cu_series = pd.read_csv(url, sep='\t')

# clean up names
col1 = cu_series.columns[0]
cu_series.rename(columns={col1: 'series_id'}, inplace=True)
cu_series['series_id'] = cu_series.series_id.str.rstrip(' ')
cu_series['series_title'] = cu_series.series_title.str.replace(', all urban consumers, seasona
cu_series['series_title'] = cu_series.series_title.str.replace(' in U.S. city average', '')

cu_series = cu_series[(cu_series.area_code == '0000')
                      & (cu_series.seasonal == 'S')
                      & (cu_series.periodicity_code == 'R')]
display(cu_series)
```

	series_id	area_code	item_code	seasonal	periodicity_code	base_code	base_period	se
0	CUSR0000SA0	0000	SA0	S	R	S	1982-84=100	
1	CUSR0000SA0E	0000	SA0E	S	R	S	1982-84=100	
2	CUSR0000SA0L1	0000	SA0L1	S	R	S	1982-84=100	
3	CUSR0000SA0L12	0000	SA0L12	S	R	S	1982-84=100	a
4	CUSR0000SA0L12E	0000	SA0L12E	S	R	S	1982-84=100	sh
...	...	...	...	...	...	...	...	
313	CUSR0000SS62032	0000	SS62032	S	R	S	DECEMBER 1997=100	A to
314	CUSR0000SS62053	0000	SS62053	S	R	S	DECEMBER 1997=100	Pe
315	CUSR0000SS62054	0000	SS62054	S	R	S	DECEMBER 1997=100	Ve
316	CUSR0000SS68023	0000	SS68023	S	R	S	DECEMBER 1997=100	pr a
317	CUSR0000SSFV031A	0000	SSFV031A	S	R	S	DECEMBER 2005=100	el s

318 rows × 13 columns

Note time series data with different start and end dates

```
# check start and end dates
df = cu_series[(cu_series.area_code == '0000')
         & (cu_series.seasonal == 'S')
         & (cu_series.periodicity_code == 'R')]
df.groupby(['end_year', 'end_period', 'begin_year', 'begin_period']).series_id.nunique().to_fr
```

				series_id
end_year	end_period	begin_year	begin_period	
2000	M05	1990	M01	1
2019	M12	2008	M01	1
2020	M12	1998	M01	1
2021	M01	2002	M01	1
	M03	1947	M01	34
		1952	M01	4
		1953	M01	7
		1956	M01	7
		1957	M01	7
		1967	M01	22
		1969	M01	4
		1976	M01	1
		1978	M01	31
		1980	M01	3
		1981	M01	2
		1983	M01	4
		1984	M01	5
		1985	M01	3
		1986	M01	2
		1987	M01	1
		1988	M01	1
		1989	M01	15
		1990	M01	5
		1991	M01	7
		1992	M01	5
		1993	M01	8
		1994	M01	7
		1995	M01	2
		1996	M01	2
		1997	M01	5

end_year	end_period	begin_year	begin_period	series_id
			M12	4
		1998	M01	38
		1999	M01	13
		2000	M01	4
		2001	M01	2
		2002	M01	7
		2003	M01	8
		2004	M01	4
		2005	M01	3
		2006	M01	8
		2007	M01	2
		2008	M01	2
		2009	M01	3
		2010	M01	16
		2011	M01	1
			M03	5

Examine CES (employment, hours, earnings) data

Column data + first 5 rows:

```
# check column info for CES table
table_ref = dataset_ref.table('employment_hours_earnings')
table = client.get_table(table_ref)
display(table.schema)

# show first 5 rows
client.list_rows(table, max_results=5).to_dataframe()
```

```
[SchemaField('series_id', 'STRING', 'REQUIRED', 'Code identifying the specific series', (), N
one),
 SchemaField('year', 'INTEGER', 'NULLABLE', 'Identifies year of observation', (), None),
 SchemaField('period', 'STRING', 'NULLABLE', 'Identifies period for which data is observed. M
01 = January, M02 = February….M13 = Annual Average', (), None),
 SchemaField('value', 'FLOAT', 'NULLABLE', 'Price index for item', (), None),
 SchemaField('footnote_codes', 'STRING', 'NULLABLE', 'Identifies footnote for the data serie
s', (), None),
 SchemaField('date', 'DATE', 'NULLABLE', 'Specifies period in date format. M01 -> [year]-01-0
1...., M12 -> [year]-12-01, M13 -> [year]-12-31, Q01 -> [year]-01-15...., Q04 -> [year]-10-1
5, Q05 -> [year]-12-31, S01 -> [year]-06-30, S02 -> [year]-12-30, S03 -> [year]-12-31, A01 ->
[year]-12-31', (), None),
 SchemaField('series_title', 'STRING', 'NULLABLE', '', (), None)]
```

	series_id	year	period	value	footnote_codes	date	series_title
0	CES0000000001	1949	M01	44668.0	None	1949-01-01	All employees, thousands, total nonfarm, seasonally adjusted
1	CES0500000001	1964	M01	47925.0	None	1964-01-01	All employees, thousands, total private, seasonally adjusted
2	CES0500000006	1972	M01	49033.0	None	1972-01-01	Production and nonsupervisory employees, thousands, total private, seasonally adjusted
3	CES0500000082	1976	M01	9281433.0	None	1976-01-01	Aggregate weekly payrolls of production and nonsupervisory employees, thousands, total private, seasonally adjusted
4	CES0600000001	1969	M01	22644.0	None	1969-01-01	All employees, thousands, goods-producing, seasonally adjusted

The ces.series table is included in the Kaggle dataset.

We are only concerned with wages, so again we'll filter the data:

- data_type_code: 11 (average weekly earnings of all employees)
- seasonal: S (seasonally adjusted)

Note: There is also data available for average hourly earnings, but I think weekly earnings are a better representation of actual wages

```
# get ces.series
query = """
SELECT *
FROM `bigquery-public-data.bls.employment_hours_earnings_series`
WHERE data_type_code IN (11)
AND seasonal = 'S'
"""

dfces_series = querytodf(query)

dfces_series['series_title'] = dfces_series.series_title.str.replace(', seasonally adjusted',
```

```
dfces_series['series_title'] = dfces_series.series_title.str.replace('Average weekly earnings

display(dfces_series)
```

```
This query will process 4660009 bytes.
```

	series_id	supersector_code	industry_code	data_type_code	seasonal	series_title	footnot
0	CES0500000011	5	5000000	11	S	total private	
1	CES0600000011	6	6000000	11	S	goods-producing	
2	CES0800000011	8	8000000	11	S	private service-providing	
3	CES1000000011	10	10000000	11	S	mining and logging	
4	CES2000000011	20	20000000	11	S	construction	
...	...	...	...	...	...	...	
597	CES8081390011	80	80813900	11	S	professional and similar organizations	
598	CES8081391011	80	80813910	11	S	business associations	
599	CES8081392011	80	80813920	11	S	professional organizations	
600	CES8081393011	80	80813930	11	S	labor unions and similar labor organizations	
601	CES8081399011	80	80813990	11	S	miscellaneous professional and similar organizations	

602 rows × 11 columns

Note different start and end dates

```
# check start and end dates
dfces_series.groupby(['end_year', 'end_period', 'begin_year', 'begin_period']).series_id.nunic
```

				series_id
end_year	**end_period**	**begin_year**	**begin_period**	
2021	**M02**	**2006**	**M03**	562
	M03	**2006**	**M03**	40

Query + Prepare Data

To keep things simple, we'll use year-over-year (YOY) percent change for our growth and inflation calculations. This means our growth rate will be updated monthly, but will represent a change over the past year.

Build queries:

```
# query for cpi data
qcpi = """
SELECT series_id, area_name, item_name, date, period, value
FROM `bigquery-public-data.bls.cpi_u`
WHERE area_code = '0000'
AND periodicity_code = 'R'
AND seasonal_code = 'S'
AND series_id IN (
    SELECT
        CASE
            WHEN EXTRACT(YEAR FROM MAX(date)) >= 2021 THEN series_id
        END
    FROM `bigquery-public-data.bls.cpi_u`
    GROUP BY series_id
)
ORDER BY area_name, series_id, item_name, date
"""

# query for ces data
qces = """
SELECT ces.series_id, ces_s.industry_code, ces.date, ces.period, ces.value
FROM `bigquery-public-data.bls.employment_hours_earnings` AS ces
INNER JOIN `bigquery-public-data.bls.employment_hours_earnings_series` AS ces_s
    ON ces.series_id = ces_s.series_id
WHERE ces_s.data_type_code IN (11)
AND ces_s.seasonal = 'S'
ORDER BY ces_s.industry_code, ces.series_id, ces.date
"""
```

Query and clean:

```
# cpi data
dfcpi = querytodf(qcpi).rename(columns = {'item_name': 'category'})
dfcpi['date'] = pd.to_datetime(dfcpi['date'])
```

```
# ces data

dfces = querytodf(qces)
dfces['date'] = pd.to_datetime(dfces['date'])

# get industry names from https://download.bls.gov/pub/time.series/ce/ce.industry
url = 'https://download.bls.gov/pub/time.series/ce/ce.industry'
legend_i = pd.read_csv(url, sep='\t')

# replace industry code with industry name
dfces.industry_code = dfces.industry_code.replace(list(legend_i.industry_code), list(legend_i.
dfces.rename(columns = {'industry_code': 'industry'}, inplace=True)
```

```
This query will process 85029891 bytes.
This query will process 286543918 bytes.
```

Check missing values

```
# check missing values
print(dfcpi.info())
print()
print(dfces.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 83542 entries, 0 to 83541
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   series_id  83542 non-null  object
 1   area_name  83542 non-null  object
 2   category   83542 non-null  object
 3   date       83542 non-null  datetime64[ns]
 4   period     83542 non-null  object
 5   value      83542 non-null  float64
dtypes: datetime64[ns](1), float64(1), object(4)
memory usage: 3.8+ MB
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 108400 entries, 0 to 108399
Data columns (total 5 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   series_id  108400 non-null  object
 1   industry   108400 non-null  object
 2   date       108400 non-null  datetime64[ns]
 3   period     108400 non-null  object
 4   value      108400 non-null  float64
dtypes: datetime64[ns](1), float64(1), object(3)
memory usage: 4.1+ MB
None
```

Confirm start and end dates

```
# confirm start and end dates cpi
df = dfcpi.groupby('series_id').agg({'date': ['min', 'max']}).reset_index()
df.columns=['series_id', 'min', 'max']
```

```
display(df.groupby(['max', 'min']).series_id.nunique().to_frame())

# confirm start and end dates ces
df = dfces.groupby('series_id').agg({'date': ['min', 'max']}).reset_index()
df.columns=['series_id', 'min', 'max']
df.groupby(['max', 'min']).series_id.nunique().to_frame()
```

		series_id
max	**min**	
2021-01-01	**2002-01-01**	1
2021-03-01	**1997-01-01**	193
	1997-12-01	4
	1998-01-01	38
	1999-01-01	14
	2000-01-01	4
	2001-01-01	2
	2002-01-01	7
	2003-01-01	8
	2004-01-01	4
	2005-01-01	3
	2006-01-01	8
	2007-01-01	2
	2008-01-01	2
	2009-01-01	3
	2010-01-01	16
	2011-01-01	1
	2011-03-01	5

		series_id
max	**min**	
2021-02-01	**2006-03-01**	562
2021-03-01	**2006-03-01**	40

Calculate inflation (growth rate of CPI)

https://en.wikipedia.org/wiki/Inflation

```
# calculate inflation for each series_id (yoy)

# def yoy(data, colval, colgroup, newcol):
#     s = pd.Series()
#     for group in data[colgroup].unique():
#         df0 = data[data[colgroup] == group]
#         s0 = df0.value.pct_change(periods=12)*100
#         s = s.append(s0)
#     df = data
#     df[newcol] = s
#     return df

def yoy(data, colval, colgroup, newcol):
    s = pd.Series()
    for group in data[colgroup].unique():
        df0 = data[data[colgroup] == group]
        s0 = df0[['date', colval]].set_index('date')[colval].pct_change(freq=pd.DateOffset(yea
        s = s.append(s0)
    df = data
    df[newcol] = list(s)
    return df

dfinf = yoy(dfcpi, 'value', 'series_id', 'inflation_yoy')

# some series have different starting dates (separate by 1997-01-01, 2012-03-01)
s = dfinf.groupby('series_id').date.min()
s97 = s[s <= pd.to_datetime('1997-01-01')]
dfinf98 = dfinf[dfinf.series_id.isin(s97.index)]

s06 = s[s <= pd.to_datetime('2006-03-01')]
dfinf07 = dfinf[(dfinf.series_id.isin(s06.index)) &
                (dfinf.date >= pd.to_datetime('2007-03-01'))
               ]

dfinf12 = dfinf[dfinf.date >= pd.to_datetime('2012-03-01')]
```

Calculate real wage growth

Real wage growth = wage growth - inflation

```
# calculate wage growth by series_id (yoy)
dfg = yoy(dfces, 'value', 'series_id', 'growth_yoy')

# calculate real wage growth

df0 = dfinf98[dfinf98.category == 'All items']
df0 = df0[['date', 'inflation_yoy']]
df1 = dfg[['series_id', 'date', 'growth_yoy']]
df2 = pd.merge(df1, df0, how='left', on='date')

dfgr = dfg # per industry
dfgr['realgrowth_yoy'] = df2['growth_yoy'] - df2['inflation_yoy']
# display(dfgr)

dfgr12 = dfgr[dfgr.date >= pd.to_datetime('2012-03-01')]
```

```
df3 = dfgr[dfgr.industry == 'Total private'] # overall
dftp = pd.merge(df3, df0, how='left', on='date')
dftpM = dftp.melt(id_vars=['series_id',
                           'industry',
                           'date',
                           'period',
                           'value'],
                  var_name='var',
                  value_name='rate')

dftpM07 = dftpM[dftpM.date >= pd.to_datetime('2007-03-01')]
dftpM12 = dftpM[dftpM.date >= pd.to_datetime('2012-03-01')]
```

Preliminary EDA

This dataset is continually updating, so charts may change month to month. The remarks I make on the data are up to date as of April 2021.

Charts are interactive, feel free to scroll/click the legend to see/compare different data

```
# helper functions

# ranked series of 'colval', grouped by 'colgroup', either 'mean' or 'median', from 'df'
def rank(df, colval, colgroup, method):
    if method == 'mean':
        s = df.groupby(colgroup)[colval].mean().sort_values(ascending=False)
    if method == 'median':
        s = df.groupby(colgroup)[colval].median().sort_values(ascending=False)
    return s

# split long text (https://community.plotly.com/t/wrap-long-text-in-title-in-dash/11419)
def split(text, width):
    split_text = textwrap.wrap(text, width=width)
    return '<br>'.join(split_text)
```

```
# plotly figure with line, bar, and box plots

def charts(df, val, group, by):
    if by == 'top':
        s = rank(df, val, group, 'mean').head(10)
        sR = s.sort_values()
    elif by == 'bottom':
        s = rank(df, val, group, 'mean').tail(10).sort_values()
        sR = s.sort_values(ascending=False)

    lcolors = px.colors.qualitative.Pastel
    num = int(np.ceil(len(s.index)/len(lcolors)))
    colors0 = lcolors*num
    colors = dict(zip(s.index, colors0))

    fig = make_subplots(
        2, 2,
        specs=[[{'colspan': 2}, None],
               [{}, {}]],
```

```
        shared_yaxes=True,
        subplot_titles=('', 'Average YOY change', 'Distributions'),
        vertical_spacing=0.15,
        horizontal_spacing=0.05
    )

    for i in s.index:
        data = df[df[group] == i]
        fig.add_scatter(
            x=data.date,
            y=data[val],
            mode='lines',
            name=i,
            legendgroup=i,
            marker_color=colors[i],
            showlegend=False,
            row=1, col=1
        )

    df0 = sR.to_frame().reset_index()
    for i in sR.index:
#         data = df[df[group] == ind].groupby(group)[val].mean()
        data = df0[df0[group] == i]
        fig.add_bar(
            y=data[group],
            x=data[val],
            orientation='h',
            name=i,
            text=round(data[val], 4),
            legendgroup=i,
            marker_color=colors[i],
#             showlegend=False,
            row=2, col=1
        )

    for i in sR.index:
        data = df[df[group] == i]
        fig.add_violin(
            y=data[group],
            x=data[val],
            orientation='h',
            name=i,
            legendgroup=i,
            marker_color=colors[i],
            showlegend=False,
            row=2, col=2
        )

    fig.for_each_trace(lambda trace: trace.update(name=split(trace.name, 31)))
    fig.update_xaxes(ticksuffix='%', row=2)
    fig.update_yaxes(ticksuffix='%', row=1)
    fig.update_yaxes(
        visible=False,
#         tickmode = 'array',
#         tickvals = list(range(len(sR.index))),
#         ticktext=[split(i, 40) for i in sR.index],
        row=2, col=1
```

```
    )
    fig.update_layout(
        height=540,
#         bargap=0.1,
        legend = dict(yanchor="middle", y=0.5, xanchor="right", x=-0.08),
#         title_text="Inflation (YoY) since 1997",
#         showlegend=False
        legend_traceorder='reversed',
    )
    return fig
```

Inflation

```
# past year
yago1 = pd.to_datetime(datetime.date.today()-datetime.timedelta(365))

data = dfinf[dfinf.date >= yago1]
val = 'inflation_yoy'
group = 'category'

fig = charts(data, val, group, 'top')
fig.update_layout(title='Inflation in the past year, Top 10 Industries')
fig.show()

fig = charts(data, val, group, 'bottom')
fig.update_layout(title='Inflation in the past year, Bottom 10 Industries')
fig.show()
```

```
# 2012 top and bottom

data = dfinf12
val = 'inflation_yoy'
group = 'category'

fig = charts(data, val, group, 'top')
fig.update_layout(title='Inflation since 2012, Top 10 Categories')
fig.show()

fig = charts(data, val, group, 'bottom')
fig.update_layout(title='Inflation since 2012, Bottom 10 Categories')
fig.show()
```

Wage Growth

```
# past year, top and bottom

data = dfgr[dfgr.date >= yago1]
val = 'growth_yoy'
group = 'industry'

fig = charts(data, val, group, 'top')
fig.update_layout(title='Wage growth in the past year, Top 10 Industries')
fig.show()

fig = charts(data, val, group, 'bottom')
fig.update_layout(title='Wage growth in the past year, Bottom 10 Industries')
fig.show()
```

```
# 2012, top and bottom

data = dfgr12
val = 'growth_yoy'
group = 'industry'

fig = charts(data, val, group, 'top')
fig.update_layout(title='Wage growth since 2012, Top 10 Industries')
fig.show()

fig = charts(data, val, group, 'bottom')
fig.update_layout(title='Wage growth since 2012, Bottom 10 Industries')
fig.show()
```

Is our purchasing power increasing or decreasing?

Purchasing power is

- increasing if the *real growth rate of wages is positive*
- decreasing if the *real growth rate of wages is negative*

Time to answer our main question. We'll answer it in 3 ways:

1. Overall Purchasing Power (Real Wage Growth)

Real growth rate = growth rate - inflation rate

Wage data is calculated from average weekly earnings of all non-farm employees in the private sector. Inflation data is calculated from CPI of all items, average of all U.S. cities.

```
data = dftpM12[dftpM12.date >= yago1]

df = data[data['var'] == 'realgrowth_yoy']
pp = round(df.rate.mean(), 2)

text = ('<i><b>For the average American, purchasing power (real wage) <br>has increased (by ab
        str(pp) +
        '%) in the past year</b></i>')

fig = charts(data, 'rate', 'var', 'top')
fig.update_layout(title='Wage Growth, Inflation, Real Wage Growth in the Past Year')
fig.update_traces(textposition='inside', row=2, col=1, selector=dict(name='realgrowth_yoy'))
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.2, y=-0.2, showarrow=False, for
fig.show()
```

```
data = dftpM12

df = dftpM12[dftpM12['var'] == 'realgrowth_yoy']
pp = round(df.rate.mean(), 2)

text = ('<i><b>For the average American, purchasing power (real wage) <br>has increased (by ab
        str(pp) +
        '% per year) since 2012</b></i>')
```

```
fig = charts(data, 'rate', 'var', 'top')
fig.update_layout(title='Wage Growth, Inflation, Real Wage Growth since 2012')
fig.update_traces(textposition='inside', row=2, col=1, selector=dict(name='realgrowth_yoy'))
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.2, y=-0.2, showarrow=False, for
fig.show()
```

Everything looks reasonable, except for a large spike in wage growth in early 2020 (onset of COVID19).

Intuitively this doesn't make sense (Why would wages increase so sharply during a recession?), but there are a number of reasons why the numbers would show such a jump (free money from the U.S. gov, low wage earners losing their jobs thus being dropped from payroll data, etc). Going into the details of this is beyond the scope of this notebook, but possibly an interesting topic for a future analysis.

For now let's just say that our numbers will probably be skewed too high.

We now have an answer to our question: The average real growth rate (YOY) of wages in the past year is a little less than 5%, and since 2012 is a little more than 1.3%

In other words, for the average American employee living in an average American city working an average non-farm private sector job, purchasing power has increased a little less than 5% in the past year, and a little more than 1.3% since 2012.

Well, does this mean we've been listening to our friends/family/selves complain for nothing? What about non-average folks with non-average jobs? Let's dive deeper.

2. Purchasing Power by Industry

```
# plotly bar plot

def bar(df, val, group):
    lcolors = px.colors.qualitative.Pastel
    num = int(np.ceil(len(df[group])/len(lcolors)))
    colors0 = lcolors*num
    colors = dict(zip(df[group], colors0))

    fig = go.Figure()
    for i in df[group].unique():
        data0 = df[df[group] == i]
        fig.add_bar(
            y=data0[group],
            x=data0[val],
            orientation='h',
            name=i,
            marker_color=colors[i],
        )
    fig.for_each_trace(lambda trace: trace.update(name=split(trace.name, 31)))
    fig.update_yaxes(
        visible=False,
    )
    fig.update_layout(
        width = 600,
        legend = dict(yanchor="top", y=1, xanchor="right", x=-0.05),
        legend_traceorder='reversed',
        xaxis_ticksuffix = '%'
    )
    return fig
```

```
# past year industries with negative pp

df = dfgr[dfgr.date >= yago1]
val = 'realgrowth_yoy'
group = 'industry'
s = rank(df, val, group, 'mean')
data = s[s<0].to_frame().reset_index()

text = '<i><b>For the average American working in the above industries, <br>purchasing power (

fig = bar(data, val, group)
fig.update_layout(title='Industries with negative real wage growth in the past year')
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.9, y=-0.17, showarrow=False, fo
fig.show()
```

```
# 2012 industries with negative pp

df = dfgr12
val = 'realgrowth_yoy'
group = 'industry'
s = rank(df, val, group, 'mean')
data = s[s<0].to_frame().reset_index()

text = '<i><b>For the average American working in the above industries, <br>purchasing power (

fig = bar(data, val, group)
fig.update_layout(title='Industries with negative real wage growth since 2012')
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.9, y=-0.17, showarrow=False, fo
fig.show()
```

A moment of silence for those of us working in document preparation services... All kidding aside, it makes sense though, since most things are being done online now.

It looks like you can keep on complaining if you work in one of those bottom industries. But how about the rest of us, do we have no right to complain about increasing price levels?

So far we've only been concerned with overall purchasing power (based on CPI, all items), but what about different item categories in the CPI? Surely the prices of different things grow at different rates?

3. Purchasing Power of Specific Items

```
# categories outpacing wage growth in the past year

df0 = dfgr[dfgr.date >= yago1]
avgWG = df0[df0.industry == 'Total private'].growth_yoy.mean()

df = dfinf[dfinf.date >= yago1]
val = 'inflation_yoy'
group = 'category'
```

```
s = rank(df, val, group, 'mean')
data = s[s>avgWG].sort_values().to_frame().reset_index()

text='<i><b>For the average American, purchasing power of <br>the above items has decreased ir

fig = bar(data, val, group)
fig.add_vline(
    x=avgWG,
    line_dash='dash',
    annotation_text='   Average wage growth<br>(past year): ' + str(round(avgWG, 2)) + '%',
    annotation_position='bottom right',
    annotation_font_size=14,
)
fig.update_layout(title='Inflation categories outpacing wage growth in the past year')
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.7, y=-0.17, showarrow=False, fo
fig.show()
```

```
# 2012 categories outpacing wage growth

avgWG12 = dfgr12[dfgr12.industry == 'Total private'].growth_yoy.mean()

df = dfinf12
val = 'inflation_yoy'
group = 'category'
```

```
s = rank(df, val, group, 'mean')
data = s[s>avgWG12].sort_values().to_frame().reset_index()

text='<i><b>For the average American, purchasing power of <br>the above items has decreased si

fig = bar(data, val, group)
fig.add_vline(
    x=avgWG12,
    line_dash='dash',
    annotation_text='   Average wage growth<br>since 2012: ' + str(round(avgWG12, 2)) + '%',
    annotation_position='bottom right',
    annotation_font_size=14,
)
fig.update_layout(title='Inflation categories outpacing wage growth since 2012')
fig.add_annotation(text=text, xref='paper', yref='paper', x=-0.9, y=-0.17, showarrow=False, fo
fig.show()
```

The good news: Most categories have gotten relatively more affordable (inflation < wage growth) in the past year and since 2012.

The bad news: There are some important categories among the ones that have gotten relatively more expensive (inflation > wage growth): things that everyone probably expected like healthcare, education, and housing costs. However there are also some common food categories like meats, eggs, and fish!

In order for an average American consumer to maintain their purchasing power of beef, they would have needed average annual salary growth of about 3.6% since 2012

```
# # WIP
# s = rank(dfgr, 'realgrowth_yoy', 'industry', 'mean')
# data = s[s<0].to_frame().reset_index()

# lcolors = px.colors.qualitative.Dark24
# num = int(np.ceil(len(data.industry)/len(lcolors)))
# colors0 = lcolors*num
# colors0
```

Final Notes

- This dataset is continually updating, so charts may change month to month. The remarks I make on the data are up to date as of April 2021.
- This analysis is not reflective of wages and inflation globally, but in the U.S. only.
- This is not an analysis of absolute purchasing power, but the change in purchasing power.
- The CPI is just one source of price measurement, there are others: https://en.wikipedia.org/wiki/Price_index
- Likewise, the CES is just one source of wage measurement, there are also others: https://www.bls.gov/bls/blswage.htm

I am still practicing, let me know if you find any mistakes! Any other suggestions/comments are welcome!

Photo by Nixon Johnson from Pexels